

DAT 4기 캡스톤 프로젝트

# 음성인식 모델과 RAG를 활용한 상호 작용 의료 챗봇 시스템 개발 : 노인 질환을 중심으로

[ 휘뚜루마뚜루 팀 ]  
최규민  
김선화  
함인규



# 목차

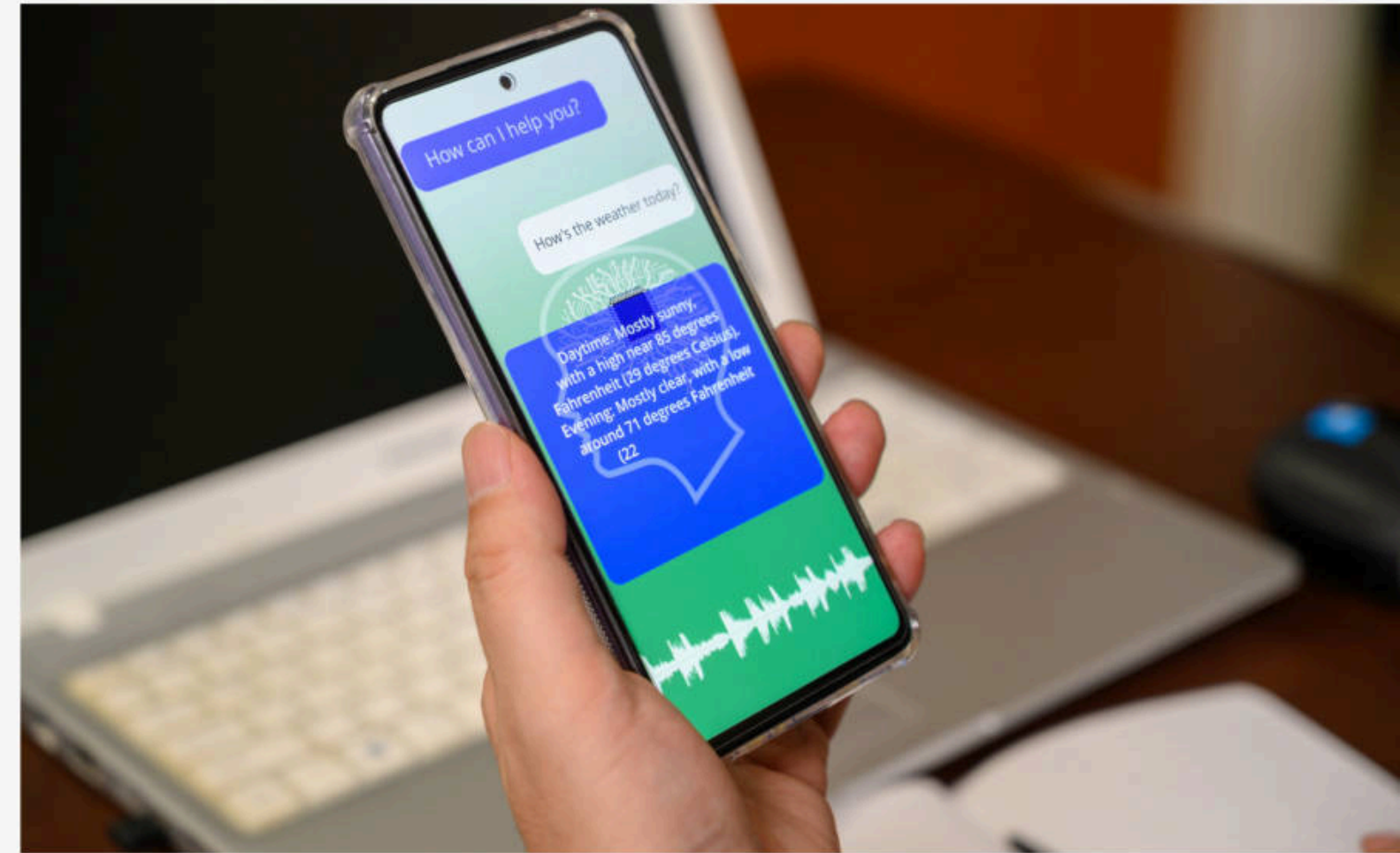
## 01. 연구 배경 및 목적

## 02. 음성인식 챗봇 시스템 구현

- 음성 인식 후 텍스트 변환 모델

- RAG 기반 구현

## 03. 결론







# 01. 연구 배경 및 목적

---

01. 연구 배경

02. 연구 목적

# 연구 배경

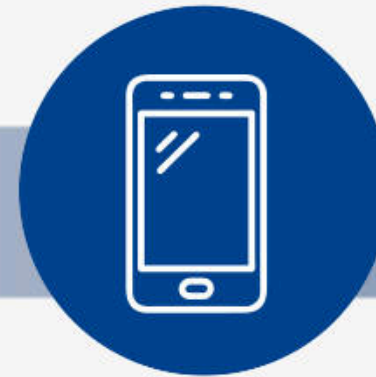


## 고령화 현상

통계청의 장래 인구 추계에 따르면,  
2025년에 우리나라 65세 이상의 고령 인구 비율이  
20.3%를 넘어 초고령 사회로 진입할 것으로 예상



인구 구조의 변화뿐만 아니라  
노인들의 건강과 삶의 질에 대한  
사회적 관심을 증가시키는 중요한 요소  
⇒ 노인들은 다양한 신체적, 정신적 질환에 노출될 위험이 높으므로,  
건강 관리 및 질병 예방을 위한 체계적 접근 필요



## 디지털 기기 사용

2023년 디지털 정보 격차 실태조사 보고서  
⇒ 고령층의 디지털 정보화 수준은 종합 70.7%



노인들이 최신 기술을 활용하여  
자신의 건강을 관리하는 데 있어  
중대한 장벽으로 작용함



## 음성인식 챗봇 시스템

노인 질환 관리에 도움을 줄 수 있는  
상호 작용 음성인식 챗봇 시스템



노인들이 텍스트 입력 대신  
간편하게 질문을 할 수 있는 환경을 제공함으로써,  
디지털 기기 사용에 대한 두려움을 줄이고,  
보다 쉽게 건강 정보를 얻을 수 있도록 도와줍니다.







## 02. 음성인식 챗봇 모델 구현

---

01. 음성 인식 후 텍스트 변환 모델

02. RAG 기반 구현

# 데이터 수집

## 데이터 수집

### 1. 질환 정보 수집

#### 주요 노인 질환

골다공증, 우울증, 암, 수면장애, 당뇨병, 고혈압성 신질환, 알츠하이머 치매, 골관절염, 뇌졸중, 백내장, 녹내장, 파킨슨병, 심근염, 부정맥

#### 자료 출처

삼성서울병원 질환백과 : 신뢰할 수 있는 의료 정보 제공

#### 활용 방안

- 문서화 : 질환 정보를 PDF 파일로 저장
- 챗봇 모델 개발 및 운영 : 질환 정보를 기반으로 한 챗봇 서비스 개발

삼성서울병원

진료예약 건강정보 진료차트 나눔소통 병원안내

건강정보 > 질환정보 > 질환백과

빠른예약 · 의료진 검색 · f · S · 인제 TIP

### [내분비/유방 질환] 당뇨병

관련진료과 삼성서울병원 내분비대사내과, 가정의학과

질환명 : [한글명] 당뇨병 [영문명] Diabetes Mellitus

#### 정의

당뇨병이란 우리 몸이 섭취한 음식을 적절하게 사용하지 못하여 혈액 속의 포도당(혈당)의 수치가 정상인보다 훨씬 높은 상태를 말하며, 우리 몸에서 에너지로 사용하는 포도당이 소변으로 빠져 나간다고 하여 이름이 붙여진 병입니다.

#### 원인

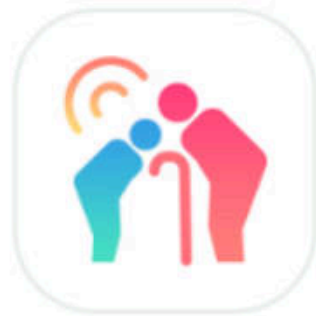
유전, 바이러스, 비만증, 노화, 식사습관, 스트레스 및 약물 등의 원인으로 생깁니다.

- 유전성향이 강하나, 양친 모두 당뇨병이라도 자녀에게서 당뇨병이 발생할 확률은 40% 정도입니다. 유전요인 보유자가 이노제, 경구피임약, 갑상선 호르몬 등의 약을 복용시 혈당상승 및 당뇨병 악화 가능성이 높아집니다.
- 콕사키 바이러스 감염 등이 췌장의 베타세포를 파괴시켜 당노를 유발 할 수 있습니다.
- 비만은 인슐린 저항성을 높혀 당노를 일으킬 수 있습니다.
- 체내의 세포기능 저하로도 당노가 유발될 수 있습니다.
- 스트레스가 지속되면 코티솔이라는 호르몬이 분비되고, 코티솔이 인슐린 작용을 방해하여 당노에 걸릴 확률이 높아집니다.

QUICK MENU > 건강의학센터 심장뇌혈관병원 암병원 양성자치료센터 진료협력/의료 인재채용 소식지신청



# 데이터 수집



한국어

자유대화 음성(노인남여)

2020 | 306.87 GB

© 11,373 ♡ 54 📄 1,310

데이터 수집

## 2. 음성 데이터 수집

- **백본 모델 : OpenAI Whisper 모델 사용**
- **목적**  
노인들의 음성을 더욱 정확하게 인식하는 음성 인식 모델 개발
- **데이터셋**  
AI-Hub의 노인남녀 자유대화 음성 데이터  
( 60세 이상, 1000명 이상 발화자, 3000여 시간 이상 음성 데이터 포함 )
- **Fine-Tuning**  
해당 데이터셋을 이용해 모델 성능 개선



# 음성 인식 후 텍스트 변환 모델

```
from transformers import WhisperProcessor, WhisperForConditionalGeneration

# Whisper 모델과 프로세서 불러오기
processor = WhisperProcessor.from_pretrained("openai/whisper-small")
model = WhisperForConditionalGeneration.from_pretrained("openai/whisper-small")
```

```
def preprocess_data(batch):
    audio = batch["audio"]
    inputs = processor(audio["array"], sampling_rate=audio["sampling_rate"], return_tensors="pt")
    batch["input_features"] = inputs.input_features[0]
    batch["labels"] = processor.tokenizer(batch["sentence"], return_tensors="pt").input_ids[0]
    return batch

# 데이터셋에 전처리 적용
dataset = dataset.map(preprocess_data)
```

## [ Fine-Tuning ]

- 모델 초기화 : Hugging Face에서 Whisper-Small 모델 API를 가져와 모델 초기화
- 입력 데이터 준비 : 학습 가능한 형태로 입력 데이터 가공

## [ 학습 데이터 전처리 과정 ]

- 오디오 데이터 로드: 학습할 오디오 데이터를 array로 로드
- 시간-주파수 도메인 변환: 오디오 신호를 시간-주파수 도메인으로 변환
- 샘플링 속도 일치: 데이터셋의 샘플링 속도를 Whisper 모델의 입력과 일치하도록 조정
- 텍스트 데이터 토큰화: 텍스트 데이터를 토큰화하여 디코더의 입력으로 변환

# 음성 인식 후 텍스트 변환 모델

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="./whisper_finetuned",
    per_device_train_batch_size=8,
    gradient_accumulation_steps=4,
    evaluation_strategy="epoch",
    learning_rate=1e-5,
    warmup_steps=500,
    num_train_epochs=3,
    save_steps=100,
    fp16=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
    tokenizer=processor.feature_extractor,
)

trainer.train()
```

## [ Whisper-Small 모델 Fine-Tuning 결과 ]

- 프로세스 : 모델 초기화, 입력 데이터 준비, 모델 학습, 학습 데이터 전처리 과정 수행
- 결과 : 노인 대상 의료 챗봇 서비스에 최적화된 음성 인식 모델 구축





# 음성 인식 후 텍스트 변환 모델

```
# 1. 음성을 녹음하여 WAV 파일로 저장하는 함수
def record_audio(filename="recording.wav", duration=5, samplerate=16000):
    print("Recording started...")
    audio = sd.rec(int(samplerate * duration), samplerate=samplerate, channels=1, dtype=np.int16)
    sd.wait()
    print("Recording finished.")

    # Save the recorded audio to a WAV file
    with wave.open(filename, "wb") as wf:
        wf.setnchannels(1)
        wf.setsampwidth(2) # 16-bit audio
        wf.setframerate(samplerate)
        wf.writeframes(audio.tobytes())
    return filename
```

## [ 서비스 적용 예시 ]

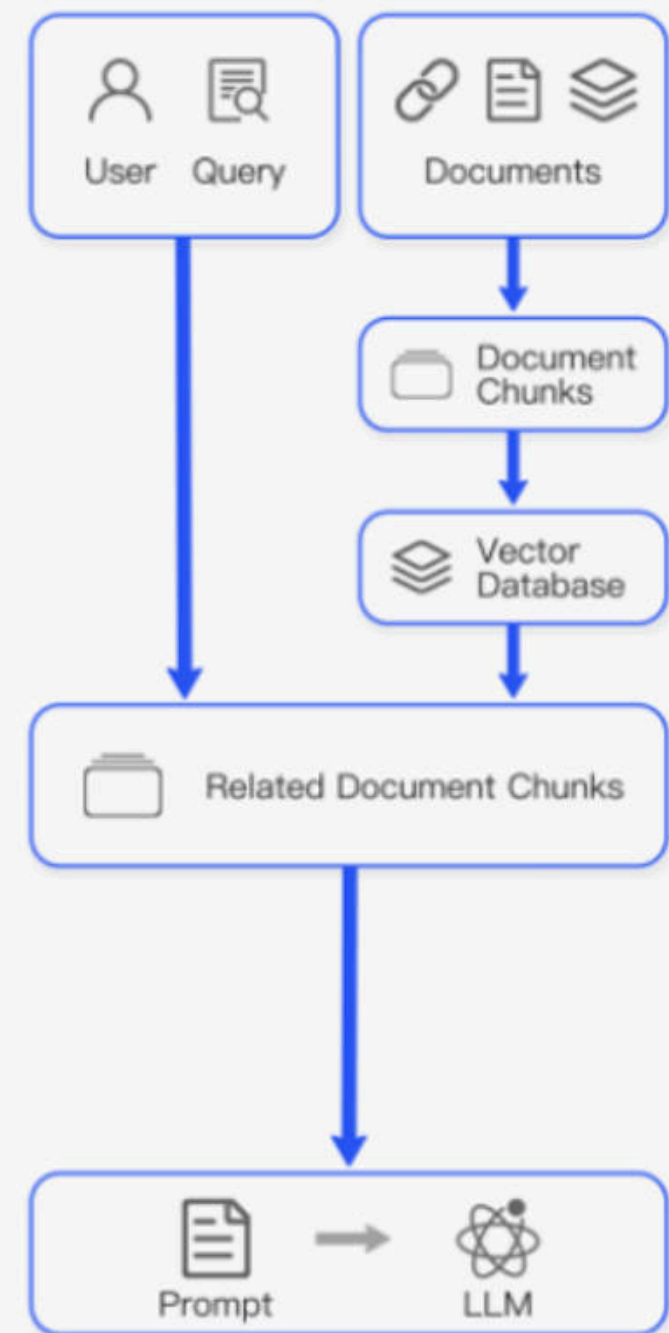
- 음성 녹음 : 위와 같은 함수를 이용해 음성을 WAV 파일 형태로 녹음
- 파일 변환 : 로컬에서 녹음한 파일을 코랩의 Whisper 모델의 입력으로 변환
- 텍스트 변환 : Whisper 모델을 이용해 음성을 텍스트로 변환
- 챗봇 입력 : 변환된 텍스트를 챗봇의 입력으로 전달

# RAG 기반 구현

## ● Architecture

### [ RAG 모델 구조 시각화 ]

1. **질의 입력:** 사용자가 챗봇에게 증상이나 아픈 부위에 대한 정보를 입력합니다.
2. **문서 준비:** 삼성서울병원의 질환 백과와 음성을 텍스트로 변환한 데이터를 문서로 준비하여 저장합니다.
3. **문서 청크 구성:** 문서를 더 작은 단위로 나누어 문서 청크를 구성합니다.
4. **벡터 변환:** 각 문서 청크를 벡터로 변환하여 벡터 데이터베이스에 저장합니다.
5. **연관 문서 청크 검색:** 사용자의 질문을 벡터로 변환하여 데이터베이스에서 연관된 문서 청크를 검색합니다.
6. **프롬프트 작성:** 검색된 문서 청크와 질의를 결합하여 프롬프트를 작성합니다.
7. **답변 생성:** 프롬프트를 대규모 언어 모델(LLM)에 전달하여 답변을 생성합니다.





# RAG 기반 구현

```
1 loader = PyPDFLoader("/content/DAT/disease_final.pdf")
2 document = loader.load()
3 document[0].page_content[:500] # 1페이지 내용 출력
```

'[눈/코/귀/인후/구강/치아]백내장질환명 : [한글명] 백내장 [영문명] cataract●정의우리 눈의 수정체는 나이가 들어가면서 자연적인 노화과정으로 본래의 투명도를 잃고 뿌옇게 변하게 되는데 이를 백내장이라고 하며, 한 연구에 따르면 50세에서 8%, 60세에는 27%, 70세에는 57%, 80세에는 80% 정도가 수술적 치료가 필요할 정도의 백내장을 가지고 있다고 합니다. 백내장은 서서히 진행되는 시력감퇴이며, 이는 안경으로도 시력이 교정되지 않는 특징이 있습니다. 만일 한쪽 눈은 정상이고, 다른 쪽 눈에 백내장이 있을 때는 백내장이 상당히 진행된 후에도 시력감소를 느끼지 못하는 경우도 있습니다.●원인가장 흔한 백내장의 원인은 나이가 들면서 자연적으로 발생하는 노인성 백내장입니다. 그 외에도 산모가 임신 초기에 풍진을 앓았거나 또는 유전적인 소인에 의해서 발생하는 선천성 백내장, 외상이나 당뇨병, 포도막염, 피부질환, 자외선 과다 노출, 부신피질 호르몬과 같은 약물의 과용, 비'

```
# 전체 텍스트를 작은 크기의 덩어리인 청크로 쪼갬다.
# 여기서는 1000글자씩 쪼갬다.
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
texts = text_splitter.split_documents(document)
```

## [ PDF 파일 텍스트 데이터 로드 & 검토 ]

- PDF 파일 로드: 'PyPDFLoader' 라이브러리를 이용하여 PDF 파일을 로드
  - 페이지 리스트 생성: 로드된 PDF 파일의 각 페이지가 리스트로 저장
  - 데이터 로드 검증: 첫 페이지의 텍스트 내용을 출력하여 정상적으로 이루어졌는지 검증
  - 내용 확인: 첫 페이지의 텍스트 내용을 500자까지 출력하여 주요 내용을 확인
- ⇒ PDF 파일의 텍스트 데이터를 로드하고 검토할 수 있으며,  
이를 바탕으로 챗봇 모델 개발 및 운영에 필요한 정보를 추출할 수 있습니다.

## [ 문서 청크화 ]

- 문서 청크화: 'CharacterTextSplitter'를 이용하여  
PDF에서 추출한 문서를 1,000글자 단위로 청크화
- 중첩 설정: 각 청크는 100글자씩 중첩되도록 설

# RAG 기반 구현

```
1 # 임베딩: 임베딩 모델로는 OpenAI를 사용
2 embeddings = OpenAIEmbeddings()
3 # ChromaDB는 일종의 Vector Database로, 청크를 여기에 저장
4 docsearch = Chroma.from_documents(texts, embeddings)
```

```
1 # LLM 설정
2 openai = ChatOpenAI(model_name="gpt-3.5-turbo",
3                       streaming=False,
4                       temperature = 0)
5
6 # RAG 설정
7 qa = RetrievalQA.from_chain_type(llm = openai,
8                                   chain_type = "stuff",
9                                   retriever = docsearch.as_retriever(
10                                     search_type="mmr",
11                                     search_kwargs={'k':3, 'fetch_k': 10}),
12                                   return_source_documents = False)
```

## [ ChromDB에 저장하는 방법 ]

- 텍스트 청크 벡터화: OpenAI의 임베딩 모델을 이용하여 텍스트 청크를 벡터화
- 데이터베이스 저장: 벡터화된 데이터를 ChromaDB에 저장

⇒ ChromaDB는 벡터화된 데이터를 효율적으로 저장하고 검색하기 위한 벡터 데이터베이스로, 이를 통해 텍스트 데이터를 빠르게 검색하고 활용할 수 있습니다.

## [ GPT-3.5-Turbo와 RAG를 활용한 MMR 기반 다중 문서 검색 기능 설정 ]

- LLM 설정: OpenAI의 'gpt-3.5-turbo'를 LLM으로 설정
- 온도 값 설정: 온도 값을 0으로 설정하여 응답의 일관성을 높임
- MMR 설정: 'docsearch'의 MMR 기능을 설정하여 검색 시 상위 3개 문서를 반환하도록 구성



# RAG 기반 구현

```

1 def chatbot():
2     print("안녕하세요! 당신을 위한 의료 챗봇입니다.")
3     print("언제든 '종료'라고 입력하시면 종료할 수 있습니다.")
4     print("=====")
5     while True:
6         body_part = input("어디가 아프신가요? (종료하려면 '종료'를 입력하세요): ")
7         if body_part.lower() == "종료":
8             print("서비스를 종료합니다. 빠른 재유를 바랍니다!")
9             break
10        while True:
11            symptom = input(f"{body_part}에 어떤 증상이 있나요? (종료하려면 '종료'를 입력하세요): ")
12            if symptom.lower() == "종료":
13                print("서비스를 종료합니다. 빠른 재유를 바랍니다!")
14                return
15        # 사용자의 입력 정보를 조합
16        query = f"{body_part}에 {symptom} 증상이 있습니다. "
17        print("=====")
18        print("\n지금 답변을 작성하고 있습니다. 잠시만 기다려주세요!\n")
19        try:
20            # invoke(): 질문과 비슷한 정보를 DB에서 찾아 LLM에게 전달
21            response = qa.invoke({"query": query})
22            # 답변 생성
23            print("=====")
24            print("답변: ")
25            if 'result' in response:
26                # 온정을 기준으로 문장 분리 및 포매팅
27                content = response['result']
28                formatted_content = "\n".join(
29                    [sentence.strip() + '.' for sentence in content.split('.') if sentence.strip()])
30            )
31            print(formatted_content)
32        else:
33            print("답변을 찾을 수 없습니다. 다시 시도해주세요.\n")
34            print("\n* 위 답변은 정확하지 않을 수 있습니다. 정확한 진단은 의사와 상담을 받으시길 바랍니다. *")
35        except Exception as e:
36            print("오류가 발생했습니다. 다시 시도해주세요.\n")
37            print(f"오류 메시지: {str(e)}")
38        # 추가 검색 여부 확인
39        print("=====")
40        more = input(f"{body_part}에 대해 추가 검색을 원하시나요? (네/아니오): ")
41        print("=====")
42        if more.lower() == "아니오":
43            break # 다음 부위로 넘어감
    
```

## [ 챗봇 스크립트 ]

- 챗봇 초기화: 사용자에게 인사말과 안내 제공
- 사용자 입력 처리: 아픈 부위와 증상 정보를 입력받아 검색 질의로 사용
- 질문 응답 처리: 'qa.invoke' 메서드를 사용해 검색 질의를 LLM 기반의  
질의응답 시스템에 전달하고, 검색 결과를 포매팅하여 출력
- 추가 검색 확인: 사용자에게 추가 검색 여부를 묻고,  
"아니오"를 선택하면 다음 부위에 대한 질문으로 넘어감
- 안내 및 종료: 각 응답 후 추가 안내 제공, 대화가 끝나면 종료 메시지 출력
- 이 챗봇은 의료 상담에 도움을 줄 수 있지만, **답변의 정확성을 보장하지 않으므로**  
**전문적인 의학적 조언이 필요하다는 점을 명시**



# RAG 기반 구현

```

1 def chatbot():
2     print("안녕하세요! 당신을 위한 의료 챗봇입니다.")
3     print("언제든 '종료'라고 입력하시면 종료할 수 있습니다.")
4     print("=====")
5     while True:
6         body_part = input("어디가 아프신가요? (종료하려면 '종료'를 입력하세요): ")
7         if body_part.lower() == "종료":
8             print("서비스를 종료합니다. 빠른 쾌유를 빕니다!")
9             break
10        while True:
11            symptom = input(f"{body_part}에 어떤 증상이 있나요? (종료하려면 '종료'를 입력하세요): ")
12            if symptom.lower() == "종료":
13                print("서비스를 종료합니다. 빠른 쾌유를 빕니다!")
14                return
15        # 사용자의 입력 정보를 조합
16        query = f"{body_part}에 {symptom} 증상이 있습니다. "
17        print("=====")
18        print("\n지금 답변을 작성하고 있습니다. 잠시만 기다려주세요!\n")
19        try:
20            # invoke(): 질문과 비슷한 정보를 DB에서 찾아 LLM에게 전달
21            response = qa.invoke({"query": query})
22            # 답변 생성
23            print("=====")
24            print("답변: ")
25            if 'result' in response:
26                # 온점을 기준으로 문장 분리 및 포매팅
27                content = response['result']
28                formatted_content = "\n".join(
29                    [sentence.strip() + '.' for sentence in content.split('.') if sentence.strip()])
30            )
31            print(formatted_content)
32        except:
33            print("답변을 찾을 수 없습니다. 다시 시도해주세요.\n")
34            print("\n* 위 답변은 정확하지 않을 수 있습니다. 정확한 진단은 의사와 상담을 받으시길 바랍니다. *")
35        except Exception as e:
36            print("오류가 발생했습니다. 다시 시도해주세요.\n")
37            print(f"오류 메시지: {str(e)}")
38        # 추가 검색 여부 확인
39        print("=====")
40        more = input(f"{body_part}에 대해 추가 검색을 원하시나요? (네/아니오): ")
41        print("=====")
42        if more.lower() == "아니오":
43            break # 다음 부위로 넘어감
    
```

## [ 챗봇 스크립트 ]

- 챗봇 초기화: 사용자에게 인사말과 안내 제공
- 사용자 입력 처리: 아픈 부위와 증상 정보를 입력받아 검색 질의로 사용
- 질문 응답 처리: 'qa.invoke' 메서드를 사용해 검색 질의를 LLM 기반의  
질의응답 시스템에 전달하고, 검색 결과를 포매팅하여 출력
- 추가 검색 확인: 사용자에게 추가 검색 여부를 묻고,  
"아니오"를 선택하면 다음 부위에 대한 질문으로 넘어감
- 안내 및 종료: 각 응답 후 추가 안내 제공, 대화가 끝나면 종료 메시지 출력
- 이 챗봇은 의료 상담에 도움을 줄 수 있지만, **답변의 정확성을 보장하지 않으므로**  
**전문적인 의학적 조언이 필요하다는 점을 명시**



# RAG 기반 코드 실행 결과

```
1 # 챗봇 실행
2 if __name__ == "__main__":
3     chatbot()
```

안녕하세요! 당신을 위한 의료 챗봇입니다.  
언제든 '종료'라고 입력하시면 종료할 수 있습니다.

=====

어디가 아프신가요? (종료하려면 '종료'를 입력하세요): 관절  
관절에 어떤 증상이 있나요? (종료하려면 '종료'를 입력하세요): 빨각게 부어

=====

지금 답변을 작성하고 있습니다. 잠시만 기다려주세요!

=====

답변:  
빨각게 부어 있는 관절은 골관절염의 증상 중 하나일 수 있습니다.  
골관절염은 관절 주변에 염증이 발생하여 관절이 붓고 빨각게 변할 수 있습니다.  
이러한 증상이 나타날 때는 정확한 진단을 위해 의사를 방문하시고, 적절한 치료 및 관리 방법을 받는 것이 중요합니다.

※ 위 답변은 정확하지 않을 수 있습니다. 정확한 진단은 의사와 상담을 받으시길 바랍니다. ※

=====

관절에 대해 추가 검색을 원하시나요? (네/아니오): 네

=====

관절에 어떤 증상이 있나요? (종료하려면 '종료'를 입력하세요): 골다공증인 것 같아

=====

지금 답변을 작성하고 있습니다. 잠시만 기다려주세요!

=====

답변:  
제가 알고 있는 정보에 따르면, 골다공증은 뼈의 밀도와 강도가 감소하여 뼈가 약해지고 쉽게 부러지는 질환입니다.  
골다공증의 주요 증상으로는 골절(특히 척추, 손목, 고관절 골절), 키가 감소, 허리 및 등 통증, 구부정한 자세(척추 후만증), 뼈가 약해져 작은 충격에도 골절이 발생할 수 있다는 것이 있습니다.  
하지만, 진단을 받고 정확한 판단을 내리기 위해서는 의사의 진료와 검사가 필요합니다.  
따라서, 골다공증이 의심된다면 즉시 의료진과 상담하여 적절한 진단과 치료를 받는 것이 중요합니다.

※ 위 답변은 정확하지 않을 수 있습니다. 정확한 진단은 의사와 상담을 받으시길 바랍니다. ※

=====

관절에 대해 추가 검색을 원하시나요? (네/아니오): 아니오

=====

어디가 아프신가요? (종료하려면 '종료'를 입력하세요): 눈  
눈에 어떤 증상이 있나요? (종료하려면 '종료'를 입력하세요): 시야가 흐려

=====

지금 답변을 작성하고 있습니다. 잠시만 기다려주세요!

=====

답변:  
시야가 흐려지는 증상은 백내장의 일반적인 증상 중 하나일 수 있습니다.  
백내장은 눈의 수정체가 탁해지면서 시력이 저하되는 질환입니다.  
이러한 증상이 지속된다면 안과 전문의를 방문하여 진단과 적절한 치료를 받는 것이 좋습니다.

## [ 동작 방식 ]

- **아픈 부위 입력:** 사용자가 '관절'과 같이 아픈 부위를 입력합니다.
- **증상 질문:** 챗봇이 해당 부위에 대한 증상을 질문합니다.

예를 들어, '빨각게 부어'와 같은 증상을 입력받습니다.

- **병명 제시:** 입력된 증상에 맞는 병명을 제시합니다.

예를 들어, '골관절염'일 가능성을 알려줍니다.

- **설명 제공:** 제시된 병명에 대한 설명을 제공합니다.





# 03. 결론

---

01. 연구 결과

02. 연구 한계점 및 제언

# | 연구 결과



## 연구 목적 및 배경

- 노인 질환 중심의 의료 챗봇 시스템 구현
- 노인 인구 증가 및 질병 복잡성 증가로 인한 의료 정보 제공 필요성 강조



## 연구 방법 및 결과

- AI Hub에서 수집한 노인 음성 데이터로 음성 인식 모델 개발
- RAG 모델로 관련 정보 검색 및 적절한 답변 생성
- 노인 음성 인식 정확도 및 의료 정보 제공 성공



## 연구 특징 및 기대 효과

- 노인들의 발음 및 억양 처리, 키보드 입력 거부감 해소 및 적극적인 정보 입력 유도
- 의료 상담 품질 향상 및 노인들의 편리한 시스템 이용 환경 조성



# | 한계점 및 제언

## 연구의 한계점

음성인식 모델의 정확도는 특정 환경에서 제한적일 수 있습니다.  
RAG 모델이 제공하는 정보의 신뢰성과 정확성을 지속적으로 검증해야 합니다.



## 개선방안

다양한 연령대의 사용자 음성에 대한 음성인식 모델의 정확도 향상 필요  
의료 전문가와의 협업을 통해 정보 신뢰성 확보 및 정기적인 업데이트 필요



## 사용자 경험 개선

사용자 피드백을 적극적으로 반영하여 시스템 개선 필요  
정기적인 설문조사나 사용자 인터뷰를 통해 피드백 수집 및 기능 강화



／  
**DAT 4기  
캡스톤 프로젝트**

# Thank you.

휘뚜루마뚜루 팀 | 최규민, 김선화, 함인규

음성인식 모델과 RAG를 활용한  
상호 작용 의료 챗봇 시스템 개발  
: 노인 질환을 중심으로